

Nguyên nhân gốc

Sau khi làm quiz lần 2 trên WordPress:

- Native LearnPress vẫn hiện bảng **Last Attempt** đúng (lấy từ meta history được ghi bởi hàm mới `custom_write_quiz_attempts_history` ở commit “lần thứ 13”).
- NextJS thì không hiện bảng vì `attemptsList` bị rỗng.

Luồng bị lỗi nằm ở 2 chỗ phối hợp:

1. **PHP** `handle_get_custom_quiz_attempts` (trong `theme-file-editor/functions.php`) vẫn trả về attempt lần 1, nhưng logic `extract result / time_spent / user_mark` từ `LP_User_Items_Result_DB` + meta **không lấy được đúng dữ liệu** của record cũ (đặc biệt khi record đó được tạo từ NextJS submit). Kết quả là attempt lần 1 bị trả về với `result_num ≈ 0` và `time_spent = "00:00:00"`.
2. **NextJS** (`LessonViewWrapper.tsx`) có đoạn filter rất chặt:

TypeScript

```
.filter((a: any) => {
  if (a.status === "started" || a.status === "in-progress") return false;
  const isZeroTime = !a.time_spent || a.time_spent === "00:00:00" || ...;
  const isZeroScore = !a.result_num || a.result_num === 0;
  return !(isZeroScore && isZeroTime); // ← attempt lần 1 bị loại
});
```

→ `sorted` chỉ còn attempt hiện tại → `previous = []` → bảng không render (đúng với vùng đỏ trống trong ảnh 1).

Logic sau khi submit từ NextJS thì dùng `sorted.slice(0, -1)` nên vẫn ổn, nhưng khi load lại trang (sau khi làm trên WP) thì đi vào `useEffect fetch` và bị filter mất.

Cách fix chi tiết

1. Sửa JS (ưu tiên – sửa ngay được)

File: `app/courses/[slug]/lessons/[lessonId]/LessonViewWrapper.tsx`

Thay toàn bộ phần xử lý trong `useEffect fetch last attempt` (khoảng dòng 450-570) bằng logic robust hơn:

```
tsx

// ... inside .then((data) => {
if (!data) return;
```

```

// Map tất cả attempts từ API
const attempts = (Array.isArray(data.attempts) ? data.attempts : [])
  .map((att: any, i: number) => {
    const resNum =
      att.result_num != null
        ? Number(att.result_num)
        : parseFloat(String(att.result || "0").replace("%", "")) || 0;

    const isPassed =
      att.status === "passed" ||
      att.graduation === "passed" ||
      resNum >= (passingGrade || 80);

    let qCount = Number(att.questions_count) || 0;
    if (qCount === 0 && quizQuestions?.length) qCount = quizQuestions.length;
    if (qCount === 0) qCount = 1;

    const totMark = att.mark != null && Number(att.mark) > 0 ? Number(att.mark) :
    qCount;
    let uMark = 0;
    if (att.user_mark != null) uMark = Number(att.user_mark);
    else if (resNum > 0) uMark = Math.round((resNum / 100) * totMark);
    else if (isPassed) uMark = totMark;

    const qCorrect = att.correct != null ? Number(att.correct) : uMark;

    return {
      user_item_id: att.user_item_id || `att-${i}`,
      status: att.status || "completed",
      graduation: isPassed ? "passed" : "failed",
      start_time: att.start_time,
      end_time: att.end_time,
      time_spent: att.time_spent || "00:00:00",
      questions_count: qCount,
      correct: qCorrect,
      wrong: att.wrong != null ? Number(att.wrong) : Math.max(0, totMark -
qCorrect),
      skipped: att.skipped != null ? Number(att.skipped) : 0,
      points: att.points || `${uMark} / ${totMark}`,
      user_mark: uMark,
      mark: totMark,
      passing_grade: att.passing_grade || `${passingGrade || 80}%`,
      result: att.result || `${resNum.toFixed(2)}%`,
      result_num: resNum,
      questionsData: att.questionsData || att.results?.questions || {}
    }
  })

```

```

        questionsdata: att.questionsdata || att.result.questions || [],
        questions: att.questions || `${uMark} / ${totMark}`,
    });
})
// CHỈ loại bỏ started / in-progress. Giữ cả những record completed dù
score/time tạm thời = 0
    .filter((a: any) => a.status !== "started" && a.status !== "in-progress");

const sorted = [...attempts].sort(
    (a: any, b: any) => Number(a.user_item_id) - Number(b.user_item_id)
);

// Current = lần mới nhất (ưu tiên data.last_attempt nếu có điểm, không thì lấy
cuối sorted)
let current: any = null;
if (
    data.last_attempt &&
    (Number(data.last_attempt.result_num) > 0 ||
        Number(data.last_attempt.user_mark) > 0 ||
        data.last_attempt.status === "completed")
) {
    // ... (giữ nguyên đoạn build current từ data.last_attempt như code cũ)
    // chỉ cần đảm bảo current.user_item_id được set đúng
}

if (!current && sorted.length > 0) {
    current = sorted[sorted.length - 1];
}

setLastAttempt(current);

// Bảng Last Attempt = tất cả lần TRƯỚC current (dùng slice giống logic sau
submit)
const previous = sorted.length > 1 ? sorted.slice(0, -1) : [];
setAttemptsList(previous);

if (sorted.length > 0 || current) {
    logQuizAttemptsNextJS(
        current ? [...previous, current] : sorted,
        activeItem?.title || activeLesson?.title?.rendered
    );
}
// ...

```

Điểm quan trọng nhất:

Đổi filter từ “loại zero-score + zero-time” → chỉ loại started / in-progress

Đổi meta từ loại zero-score + zero-time / on loại started / in-progress.

Và luôn dùng `sorted.slice(0, -1)` thay vì filter theo `user_item_id` (tránh mismatch type string/number).

2. Sửa PHP (nên làm để data đúng)

Trong `theme-file-editor/functions.php`, hàm `handle_get_custom_quiz_attempts`:

- Ở đoạn cuối (sau khi build `$attempts_list`), thay vì chỉ lấy `end($completed_attempts)` làm `last_attempt`, hãy dùng logic giống `custom_build_quiz_previous_attempts_history` để lấy đúng score/time của mọi record.
- Hoặc đơn giản hơn: sau khi có `$attempts_list`, nếu có attempt cuối cùng thì đọc meta history đã được sync (`attempts`, `_attempts`, `_lp_quiz_retake_items` ...) từ `user_item_id` mới nhất và merge vào.

Ví dụ nhanh (thêm sau dòng filter `$attempts_list`):

PHP

```
// Ưu tiên lấy history đã được ghi bởi custom_write_quiz_attempts_history
if (!empty($attempts_list)) {
    $latest_uid = end($attempts_list)['user_item_id'] ?? 0;
    if ($latest_uid) {
        $history_keys = ['attempts', '_attempts', '_lp_quiz_retake_items',
            '_lp_retake_items'];
        foreach ($history_keys as $hkey) {
            $hist = learn_press_get_user_item_meta($latest_uid, $hkey, true);
            if (is_array($hist) && !empty($hist)) {
                // Merge / replace $attempts_list bằng $hist nếu đầy đủ hơn
                // (bạn có thể map lại format cho khớp)
                break;
            }
        }
    }
}
```

Sau đó `last_attempt = end($attempts_list)` và trả về như cũ.

3. Kiểm tra sau khi fix

1. Xóa cache / hard refresh trang quiz trên NextJS.
2. Làm quiz lần 1 trên NextJS → thấy 100%, không có bảng Last Attempt (đúng vì chưa có lần trước).

3. Làm lại lần 2 trên WordPress → refresh NextJS → bảng **Last Attempt** phải hiện đúng dòng 3/4 – 00:00:11 – 75% như native.

4. Mở DevTools → Network tab xem response của `/api/user/quiz-attempts?quiz_id=...` phải có `attempts` chứa ≥ 2 phần tử và `last_attempt` là lần mới nhất.

Sau khi áp dụng 2 thay đổi trên (đặc biệt là nơi filter + dùng `slice(0, -1)`), bảng Last Attempt sẽ đồng bộ đúng với WordPress.